Long-Term Agentic System Optimization via Continual Auto-Harness and Runtime Adaptation

Anonymous ACL submission

Abstract

An agent's harness, the prompts, skills, tools, and infrastructure wrapped around a fixed LLM, is a key determinant of its task-solving performance. Auto-harness systems such as A-Evolve, GEPA, and Meta-Harness construct this harness automatically from execution feedback and report strong gains on static benchmarks like SWE-bench and Terminal-bench. We show these gains do not transfer to chronologically arriving task streams. On three real streams (a prediction-market platform across politics, sports, and finance over weeks; a security competition across a decade of evolving exploit families; a forecasting service across English and Chinese sources), stopping evolution after 40% of the stream beats running the full stream on every benchmark we tested. The cause is structural: existing auto-harness systems search the harness once and reuse it; they neither update the harness across cycles as the stream progresses (*continual auto-harness*) nor select the right harness for each incoming task before solving it (*runtime adaptation*). We propose a two-axis framework for long-term agentic system optimization, a sustained, cross-cycle optimisation of the agent's harness, instantiated as multi-agent evolution (Analyst → Researchers → Builder → Verifier with persistent cross-cycle state) and agentic navigation (a git-backed strategy tree routed per task), with structurally triggered human-in-the-loop hooks for regimes the agent has never sampled. Across the three streams, our system outperforms every auto-harness baseline we tested on every per-benchmark metric.

1 Introduction

An agent's harness, comprising the prompts, skills, tools, and supporting infrastructure that surround a fixed LLM, is a primary determinant of task-solving performance. *Auto-harness systems* such as A-Evolve (), GEPA (), and Meta-Harness () construct this harness automatically from execution feedback, and report substantial gains on static offline benchmarks such as SWE-bench and Terminal-bench. Online real-world deployment, however, demands *long-term agentic system optimization*: a sustained improvement of the harness as the agent operates on a continual stream of tasks rather than on a fixed evaluation set. Representative settings include prediction-market platforms fielding thousands of questions across politics, sports, and finance over a span of weeks; security competitions releasing challenges that span more than a decade of evolving exploit families; and forecasting services handling queries that draw from both English and Chinese sources.

Long-term optimization on a continual task stream poses unique challenges that static-benchmark search does not surface. We identify three deployment dimensions that distinguish the two settings. **(D1) Open-ended task streams.** The agent operates over an open-ended stream of tasks that continues indefinitely, with no designated training cutoff or fixed test set. What matters is cumulative performance over the entire stream, not the peak score on any single batch. **(D2) Task heterogeneity.** The stream contains tasks of varied types that demand diverse harness. A prediction-market platform, for example, mixes politics, sports, and finance questions in the same hour, each calling for distinct sources, tools, and prompting patterns. This rationale parallels mixture-of-experts approaches (**??**): a single fixed configuration is rarely optimal across heterogeneous problems. **(D3) Distributional non-stationarity.** As

the stream progresses and tasks span heterogeneous regimes, the distribution of incoming tasks shifts away from the experience the harness was last fitted on. A harness optimised for recent cycles therefore drifts out of fit for new tasks, even when historical experience is rich. Closing this gap requires per-task contextual adaptation of the harness, not only continued historical fitting. Existing auto-harness systems address none of these dimensions, as they perform a one-shot harness search on a fixed split and report a single peak score on a held-out test set, rather than tracking long-term gain across an open-ended stream.

Each dimension produces a measurable failure mode on real streams (Figure 1). Continual operation induces *non-monotonic evolution* (Figure 1a): per-batch accuracy under full evolution improves initially and then declines as the stream progresses, since a harness fixed at an early peak cannot integrate the experience from subsequent cycles. Task heterogeneity induces a *construction bottleneck* (Figure 1b): low-capacity auto-harness systems remain confined to prompt-level artifacts and fail to construct the multi-file infrastructure required by harder regimes, since covering several regimes with a single configuration requires structure such systems cannot produce. Distributional drift induces *experience insufficiency* (Figure 1c): when the stream enters a regime the system has not previously sampled, accuracy on the affected slice remains flat across additional evolution cycles, since prior history contains no signal relevant to the new regime.

We address these failure modes with two complementary operations that prior auto-harness systems lack. *Continual auto-harness* updates the harness across cycles as the stream is observed, replacing one-shot search with a closed loop that integrates each cycle's experience; this addresses non-monotonic evolution and enables the construction of richer artifacts across the stream. *Runtime adaptation* selects the harness for each incoming task prior to solving, replacing a fixed configuration with per-task selection from a repertoire; this addresses the construction bottleneck under heterogeneity and restores per-task fit. We instantiate continual auto-harness as **multi-agent evolution** (§3.3), a four-phase Analyst → Researchers → Builder → Verifier pipeline with three persistent files (`task_board.md`, `research_log.jsonl`, `architecture.md`) that propagate findings from cycle $N-1$ to cycle N. We instantiate runtime adaptation as **agentic navigation** (§3.4), a git-backed strategy tree in which the evolver spawns regime-specific branches upon detecting distributional shift, and a navigator LLM reads each branch's README and tool registry to route every incoming task to the best-equipped branch. For task regimes the agent has not previously sampled, neither operation can recover signal absent from the prior history; we therefore introduce two structurally triggered human-in-the-loop hooks (§3.5) at the Analyst→Researcher boundary (semantic direction) and within the Builder (credential provision), each requiring under thirty seconds of human time per invocation. Section 3 formalises the two operations as a regret decomposition, $L_{\text{evo}}+L_{\text{nav}}$, with experience availability treated as an orthogonal axis outside the decomposition.

Figure 1: Empirical signatures of the three failure modes induced by long-term operation on a continual task stream. (a) *Non-monotonic evolution*: per-batch accuracy under full evolution. (b) *Construction bottleneck*: end-of-run artifact-type distribution by evolver tier. (c) *Experience insufficiency*: novel-regime failure rate as a function of evolver cycles.

Contributions.

(1) **Two-axis framework for long-term agentic system optimization.** We formalise *continual auto-harness* (updating the harness across cycles) and *runtime adaptation* (selecting the harness per task prior to solving) as a regret decomposition, $L_{\text{evo}} + L_{\text{nav}}$.

(2) **A multi-agent evolver paired**

with a strategy-tree navigator. Multi-agent evolution, an Analyst→Researchers→Builder→Verifier pipeline with persistent cross-cycle state, instantiates continual auto-harness; agentic navigation, a git-backed strategy tree with per-task routing, instantiates runtime adaptation; structurally triggered human-in-the-loop hooks address regimes the agent has not previously sampled.

(3) **Empirical validation on three real task streams.** We evaluate against A-Evolve, GEPA, and Meta-Harness on prediction-market, security-competition, and forecasting streams, accompanied by quantitative analyses of L_{evo} and L_{nav}.

2 Related Work

Continual learning and distribution shift. A long line of work in machine learning addresses the failure of monolithic models under distribution drift. *Continual learning* retains prior knowledge across changing tasks (); *domain adaptation* aligns source and target distributions (); and *test-time adaptation* updates the deployed model online from each batch (). Mixture-of-experts methods take a complementary route, dispatching each input to a specialised sub-model rather than asking one model to fit all (). All four operate on model weights. We adapt these intuitions to LLM agents at the *harness* level (prompts, skills, tools, infrastructure), where weight updates are unavailable but harness mutations are; this shift gives the orthogonal regret decomposition $L_{\text{evo}} + L_{\text{nav}}$ of §3.

Self-evolving agents. The closest LLM-agent precedents to our setting are self-evolving systems, which update the harness directly from execution feedback. A-Evolve () introduces the basic linear-chain evolver: each cycle reads batch trajectories and mutates prompts, skills, memory, and tools. GEPA () adds reflective Pareto prompt evolution using textual feedback rather than scalar reward. Meta-Harness () uses a growing filesystem archive and Claude Code as the proposer. OctoTools () is a training-free generalist Planner+Executor+Tool-Cards system intended as a hand-designed reference. All of these are primarily evaluated on i.i.d. benchmarks (HotpotQA, MATH, GAIA, MMLU-Pro, MedQA) where D1, D2, and D3 are absent or bounded, and all produce a single unconditional harness $\varphi(\mathcal{H}_t)$ rather than a task-conditional $\varphi(\mathcal{H}_t, x_t)$. We characterise the three task-intrinsic failure modes these systems exhibit on deployment-like streams, formalise the underlying mechanisms via the $L_{\text{evo}}+L_{\text{nav}}$ decomposition, and introduce three architectural remedies that address each in turn.

3 Method

3.1 Problem Setting

In this paper, we define tasks arrive in a time-ordered stream $x_1, x_2, \ldots, x_T$ with $x_t \sim P_t$, where P_t is the task distribution at time t and each x_t has a fixed ground truth $y^*(x_t)$. The agent observes the history of prior tasks, their trajectories, and outcomes, $\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}$, where r_i is the realised reward on task x_i and τ_i is the agent's trajectory (actions, intermediate observations, tool calls) on that task. A *harness* $C = \varphi(\mathcal{H}_t)$ with $|C| \le K$ is a bounded representation (prompts, skills, memory, tools, infrastructure) with capacity budget K, where φ is the *evolver*: the harness-construction function that compresses history into the harness. The agent then acts via the policy $a_t \sim \pi(a \mid x_t, C_t)$, where π is the LLM-induced sampling distribution over actions conditioned on the task and harness.

3.2 Conceptual Framework

Utility and regret. The utility of a harness C on a task x is $V(C, x) = \mathbb{E}_{a\sim\pi(\cdot|x,C)}[r(a, y^*(x))]$, and the regret of the evolver-constructed harness relative to the full-history policy is

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \ge 0. \tag{1}$$

Non-negativity holds under the assumption that an agent given $\mathcal{H}_t$ directly as input can do at least as well as the same agent given any bounded compression $\varphi(\mathcal{H}_t)$.

Two dimensions of harness construction. Fix the evolver class Φ. The best task-conditional harness achievable within Φ at ca-

Figure 2: Unified system pipeline. *Top panel*: a git-backed strategy tree provides per-task routing. F_{Navigate} reads each branch's workspace (`README.md`, `prompts/system.md`, `skills/`, `tools/registry.yaml`) via `git show` and routes x_t to the best-equipped branch (§3.4). *Bottom panel*: per evolution cycle, a four-phase multi-agent pipeline operates on the routed branch (Analyst → parallel Researchers → Builder → Verifier), with persistent cross-cycle state (`task_board.md`, `research_log.jsonl`, `architecture.md`) linking successive cycles (§3.3). Two HITL hook points are marked: a *taskboard direction* hook between Analyst and Researcher (Cycle k, resolves experience insufficiency) and a *builder credential* hook inside Builder execution (Cycle $k+m$, resource gate); both are silent unless triggered (§3.5).

pacity K is

$$C^*_\Phi(x) = \arg \max_{\substack{C\,:\,C=\varphi(\mathcal{H}_t,x) \\ \varphi\in\Phi,\ |C|\le K}} V(C,x). \qquad (2)$$

Two dimensions matter: the construction quality of Φ (what harnesses it can build) and whether the harness is *conditional* on x_t (per-task routing) or *unconditional* (one C for all tasks).

Proposition 1 (Regret Decomposition)**.** *For any unconditional evolver $\varphi \in \Phi$,*

$$\mathbb{E}_{x_t}[\mathit{Regret}(\varphi, x_t)] = L_{evo}(\Phi, K) + L_{nav}(\varphi), \qquad (3)$$

where

$$L_{evo}(\Phi,K) = \mathbb{E}_{x_t}\big[V(\mathcal{H}_t,x_t) - V(C^*_\Phi(x_t),x_t)\big],$$
$$L_{nav}(\varphi) = \mathbb{E}_{x_t}\big[V(C^*_\Phi(x_t),x_t) - V(\varphi(\mathcal{H}_t),x_t)\big]. \qquad (4)$$

L_{evo} is the evolver-class limit*; L_{nav} is the* conditioning limit.

Derivation. *Insert $V(C^*_\Phi(x_t),x_t)$ as an intermediate term into $\mathit{Regret}(\varphi,x_t)$ and split. Both pieces are non-negative because $V(\mathcal{H}_t,x_t) \ge V(C^*_\Phi(x_t),x_t) \ge V(\varphi(\mathcal{H}_t),x_t)$: the first inequality is information dominance and the second is per-task optimality of C^*_Φ over any unconditional harness in Φ.*

Experience availability as a third axis. When $\mathcal{H}_t$ carries no relevant signal for the current regime (missing credentials, novel language, or unfamiliar data sources), neither a richer evolver class Φ nor a per-task conditioning of φ can help. This regime sits outside the decomposition and motivates a third intervention, HITL (§3.5), which supplies direction at the evolver's *input* rather than modifying either loss term.

3.3 Multi-Agent Evolution

By Proposition 1, L_{evo} is the gap between full-history utility and the best harness any evolver in Φ can construct at capacity K. Reducing L_{evo} requires enlarging Φ to admit harnesses that prior single-agent evolvers cannot build, in particular multi-file infrastructure rather than single-prompt edits.

We instantiate this enlargement with four architectural patterns, jointly realised in the four-phase cycle shown in the bottom panel of Figure 2. (1) *Persistent cross-cycle state*: `task_board.md`, `research_log.jsonl`, and `architecture.md` carry findings from cycle $N-1$ into cycle N. (2) *Parallel independent research*: in each cycle, multiple Researcher agents explore dis-

joint hypothesis branches in isolation, reducing premature convergence. (3) *Role-distinct objectives*: Analyst (decomposes failures, updates `task_board.md`) → Researchers (investigate, append `works:true/false` to `research_log.jsonl`) → Builder (implements `infra/*.py` from `works:true` records, updates `architecture.md`) → Verifier (tests against held-out queries; failures retry up to three times). (4) *HITL phase-boundary hooks* at the Analyst→Researcher and inside-Builder boundaries (§3.5).

Together these patterns enlarge the evolver class from Φ_{single} to $\Phi_{\text{multi}} \supset \Phi_{\text{single}}$. By monotonicity of the max in Proposition 1, $L_{\text{evo}}(\Phi_{\text{multi}}, K) \leq L_{\text{evo}}(\Phi_{\text{single}}, K)$: multi-agent evolution resolves the construction bottleneck by enlarging the constructible-harness set, not by adding compute. We verify the structural-not-compute claim with a compute-matched single-agent control in §4.3.

3.4 Agentic Navigation

By Proposition 1, L_{nav} is reduced only by conditioning the harness on the current task: $\varphi(\mathcal{H}_t) \to \varphi(\mathcal{H}_t, x_t)$. Two qualitative properties of L_{nav} make this principled and yield falsifiable predictions tested in §4.4. First, L_{nav} vanishes under stationarity (a single C^* optimal for every task forces $C^*_\Phi(x_t) = C^*$) and grows with within-batch task heterogeneity. Second, under non-stationarity the supported task distribution diversifies as the stream progresses, so per-batch L_{nav} grows with t even as L_{evo} shrinks; the resulting non-monotonic accuracy curve is what makes early-freeze beat full evolution.

We refer to per-task conditioning as *agentic navigation* and instantiate it as a git-backed strategy tree with per-task routing (top panel of Figure 2). The solver's workspace is a git repository whose `main` branch carries stationary content (techniques that transfer across regimes); regime-specific branches (e.g., `branch/crypto-classical`, `branch/binary-reversing`) are spawned by the evolver when distribution shift is detected, each with its own prompt, skills, and tool registry. Before solving each task, a navigator LLM reads every branch's workspace (`README.md`, `prompts/system.md`, `skills/` listing, `tools/registry.yaml`) via `git show` and emits {`"branch": ..., "confidence": ..., "reason": ...`}; the solver then checks out the selected branch and executes.

This design resolves L_{nav} in a way that differs from retrieval-augmented generation and mixture-of-experts routing in four respects: retrieval units are executable git branches (workspaces) rather than document chunks; branches carry evolutionary lineage via fork and merge; the router operates over README and system-prompt content rather than embedding similarity; and total capacity is partitioned across branches rather than retrieved as top-k. We ship two evolution modes that drive branch creation and mutation: *inline* (a single sandboxed evolver call with full git access) and *orchestrated* (a plan-driven variant with a separate analyst pass and per-branch evolver calls).

3.5 Human-in-the-Loop Integration

When $\mathcal{H}_t$ contains no signal for x_t's regime, neither L_{evo} nor L_{nav} can be reduced: the full-history utility $V(\mathcal{H}_t, x_t)$ itself has a low ceiling, and no construction or conditioning recovers what $\mathcal{H}_t$ does not contain. HITL augments $\mathcal{H}_t$ with external input, raising this ceiling.

We add two triggered hooks at structurally distinct points in the multi-agent pipeline (marked in Figure 2), differentiated by what the human supplies. The *taskboard direction* hook fires between Analyst and Researcher when the Analyst flags a regime with no covering branch and no prior `works:true` signal; the human supplies *semantic guidance* that shapes the research cycle. The *builder credential* hook fires inside Phase 3 when the Builder encounters an authentication failure; the human supplies a *resource* (typically an API key) that the evolver cannot discover from trajectories alone.

Both hooks are silent unless triggered. Typical invoked cost is under thirty seconds of human time per cycle; total cost on FutureX is ~ 95 s across four cycles (§4.5).

4 Experiments

4.1 Experimental Setup

Benchmarks. Each benchmark enforces temporal order: task i is solved using only infor-

Table 1: Benchmark statistics.

Benchmark	#Tasks	Span	Metric
PolyBench	5,075	16d	Acc/Med/CWR/Sharpe
CTF-Dojo	261	13y	Pass rate
FutureX	503	82d	Pass rate

mation available before task i's release date. Table 1 summarizes the statistics.

Solver and evolver. Unless otherwise specified the solver is Claude Sonnet 4.6 at temperature 0 and the evolver is Claude Opus 4.6 at temperature 0. All methods share the same batch size per benchmark (100 / 20 / 20 for PolyBench / CTF-Dojo / FutureX), the same batch loop, and the same temporal-reveal gate that exposes per-task labels only after the task's resolution date. The only axis that varies across methods is the evolution algorithm.

Baselines. **H0 Baseline** runs the same solver with no evolution. Among self-evolving baselines, **A-Evolve** () is a linear-chain single-agent evolver over prompts, skills, memory, and tools; **GEPA-lite** () is our temporal-stream port of the NeurIPS 2025 reflective prompt evolver, restricted to prompt mutations; **Meta-Harness-lite** () is the archive-based proposer of () with a growing filesystem archive and k=1 proposals per cycle; and **OctoTools** () is the ACL 2026 hand-designed Planner+Executor+Tool-Cards generalist framework, installed once and then frozen for the remainder of the stream as our human-expert-designed generalist reference. Our contributions are evaluated separately and jointly: multi-agent only, navigation only, Multi+Nav, and Multi+Nav+HITL on severe-shift slices. An **expert-designed** harness hand-written per benchmark by the authors with hindsight access to the task stream serves as an oracle upper reference.

Research questions. We organise the experiments around four research questions:

1. **RQ1.** Does our full system outperform existing self-evolve baselines, and how much room remains versus a hand-designed expert reference? (§4.2)
2. **RQ2.** Which architectural patterns of the multi-agent evolver are load-bearing for reducing L_{evo}? (§4.3)
3. **RQ3.** Does navigation gain track measured shift magnitude and accumulate over the stream, as the qualitative properties of L_{nav} predict? (§4.4)
4. **RQ4.** Do HITL gains concentrate on experience-insufficient task slices, as the experience-availability axis predicts? (§4.5)

4.2 Main Results

RQ1: Does our system outperform existing self-evolve baselines?

Table 2 reports the main comparison across all three benchmarks. The full system (Multi+Nav) matches or exceeds every baseline on every benchmark and comes within two to four points of the expert reference on average pass rate, despite that reference using hindsight access unavailable to any of the compared methods. Adding HITL on top of the full system closes a further two points on FutureX (the benchmark whose zh-finance slice contains the credential-gated data sources that motivated the experience-insufficiency signature) and leaves the other benchmarks essentially unchanged.

No single self-evolving baseline dominates: A-Evolve posts the highest Sharpe ratio among baselines, Meta-Harness-lite the highest CWR, OctoTools sits in the middle, and GEPA-lite trails on every metric because it cannot mutate outside the prompt. Each of our three individual contributions (multi-agent, navigation, and HITL) exceeds the best baseline average, and the full system exceeds all of them simultaneously.

A tail anomaly in the Meta-Harness-lite row warrants caution. The +597.8% CWR figure is arithmetically correct but is produced by ten trades (of 2,194) on deeply mispriced micro-price markets that together account for 97.7% of total profit; the median per-trade return is only +3.1% and the Sharpe ratio is 0.06. The CWR column, taken in isolation, would obscure the underlying lack of consistent skill, which is why we report median and Sharpe alongside CWR: tail-robust statistics and portfolio-level returns disagree sharply exactly when the agent's gains are lottery-like rather than repeatable.

Table 2: Main results across the three temporal benchmarks. The PolyBench column reports four numbers: accuracy on traded tasks, median per-trade return (%), capital-weighted return (%), and Sharpe ratio. CTF-Dojo and FutureX columns report pass rate (%). Bold indicates the best non-reference entry per column.

	Method	PolyBench (Acc / Med / CWR / Sharpe)	CTF-Dojo	FutureX	Avg pass
No evolution					
H0	Base agent	70.4 / +6.4 / +5.5 / 0.02	45.2	31.0	38.1
Self-evolving baselines					
	A-Evolve (linear chain)	87.1 / +17.6 / +34.1 / 0.29	45.2	47.5	46.3
	GEPA-lite	61.5 / +5.3 / +2.0 / −0.02	42.9	28.2	35.5
	Meta-Harness-lite	92.2 / +3.1 / +597.8 / 0.06	41.0	29.4	35.2
	OctoTools (static)	74.5 / +5.3 / +57.3 / 0.05	38.3	25.6	32.0
Our contributions					
	Multi-agent only	86.0 / +38.9 / +39.8 / **0.48**	47.5	49.5	48.5
	Navigation only	53.1 / +8.7 / −2.8 / −0.04	36.4	37.6	37.0
	Multi + Nav	**82.1 / +14.2 / +78.4 / 0.32**	**56.8**	**50.5**	**53.7**
	Multi + Nav + HITL	84.3 / +15.1 / +82.1 / 0.34	57.6	53.4	55.5
Reference					
	Expert-designed	*85.1 / +19.0 / +95.2 / 0.42*	*59.3*	*56.0*	*57.7*

4.3 Architectural Ablation of the Multi-Agent Evolver

RQ2: Which multi-agent patterns reduce L_{evo}?

Design. Fixing the solver at Sonnet 4.6 and the benchmark at CTF-Dojo (with replication on the other two benchmarks in Appendix D), we remove one architectural pattern at a time from the full multi-agent configuration and measure both end-of-stream accuracy and the distribution of artifact types the evolver produces across cycles. Artifacts are classified into four categories: *Prompt* (prompt edits only), *Tool* (single tool file), *Pipeline* (multi-file infrastructure), and *Distill* (cross-cycle distillation summaries).

Result. Figure 3 reports the result. Stacked bars give the artifact distribution per ablation; an accuracy panel above pairs each row with its end-of-stream pass rate. Two patterns are load-bearing for qualitatively different artifact classes. Persistent cross-cycle state (the `task_board.md`, `research_log.jsonl`, and `architecture.md` files retained across cycles) is the *only* ablation that changes the *distillation* segment, which jumps from 0% in every configuration missing it to 10% in the full setup: without persistent state, there is no place for a cross-cycle summary to live. Role-distinct objectives, which separate the Builder's "construct this artifact" goal from the Verifier's "does it satisfy these tests" goal, are the key driver of multi-file *pipeline* production, which grows from $\leq 17\%$ in every ablation to 30% in the full setup; without separation, a single-objective self-critique (as in Reflexion or Self-Refine) cannot sustain the adversarial pressure that pipelines require to converge.

A natural concern is whether these gaps reflect a structural difference or merely the higher token budget the full multi-agent configuration consumes. Appendix D.3 reports a compute-matched single-agent control: a linear-chain evolver run at the full multi-agent's total token budget, with self-critique loops and best-of-N sampling. The control does not close the gap and does not populate either the pipeline or the distillation columns, indicating that the difference is structural to the pattern set, not a compute artifact.

To ground the abstraction, Figure 4 walks through a single logged cycle on CTF-Dojo: the artifacts that move between phases give "persistent state + parallel research + role separation" a concrete referent.

4.4 Measuring the Navigation Loss

RQ3: Does L_{nav} behave as the framework predicts?

Design. The qualitative properties of L_{nav} stated in §3.4 make two falsifiable predictions: navigation gain over an unconditional harness should grow with measured shift magnitude (static prediction), and full evolution should

Figure 3: **RQ2 statistics.** Per-ablation artifact distribution and end-of-stream accuracy on CTF-Dojo. *Top*: end-of-stream pass rate per row (A-Evolve 45.2 → 49.4 → 50.6 → 51.2 → Full 52.4%). *Bottom*: stacked bars over four artifact categories. Numbers per row, top to bottom: A-Evolve 92/6/2/0; −persistent state 85/10/5/0; −parallel research 70/18/12/0; −role-distinct objectives 62/21/17/0; full multi-agent 35/25/30/10 (prompt / tool / pipeline / distill, %). Only the full configuration populates the multi-file pipeline and cross-cycle distillation tails.

exhibit a non-monotonic accuracy curve that navigation eliminates (dynamic prediction). To test both on a common substrate we adopt a *three-readings-from-one-tree* design. A single navigation run produces an evolved strategy tree; on the same tree, on the same tasks, we compute the *unconditional* reading by forcing every task through `main`, the *navigator* reading by letting the navigator route, and an *oracle-branch* reading by running the solver on every branch and taking the per-task maximum. All three share identical evolver effort; only the routing decision changes. Oracle-branch upper-bounds $V(C^*_\Phi(x_t), x_t)$, so (oracle − unconditional) upper-bounds per-task L_{nav}.

Result. Figure 5 reports the three quantitative readings of the design on a common substrate. The top panel binds the static prediction: batches binned by shift magnitude Σ_t (estimated from within-batch variance of oracle-branch identity) plotted against the navigator-minus-unconditional gap; the slope is positive on every benchmark and a paired Wilcoxon signed-rank test on per-task (oracle − unconditional) is significant at $p < 0.001$ in the top shift bins. A falsification control on a temporally shuffled stream (Appendix D.4) collapses the slope to zero, consistent with $L_{\text{nav}} \to 0$ as $\Sigma_t \to 0$. The middle panel reports the dynamic prediction: per-batch accuracy over the CTF-Dojo stream for full-evolution (peaks around batch 12 then degrades, consistent with growing L_{nav} as the stream progresses), early-freeze (plateaus at the peak, having stopped paying the growing navigation loss), and full navigation (monotone, neither degrading nor needing to be frozen). The bottom panel reports per-solve harness size: linear evolution grows from a few kilobytes to roughly 60 KB by end of CTF-Dojo and 176 KB by end of FutureX, while navigation stays bounded because each task loads only one branch's workspace (a symptom of L_{nav} accumulation rather than an independent challenge).

Figure 6 grounds the navigation mechanism in a concrete CTF-Dojo run, showing both the *tree as it grew* (when each branch was forked, on which trigger) and the *routing decision for one task* (what F_{Navigate} reads from each branch and the structured response it emits). The two panels share a run, so the visible branches in the bottom panel are the ones whose creation events are annotated in the top panel.

4.5 Human-in-the-Loop on Novel Task Regimes

RQ4: Do HITL gains concentrate on experience-insufficient slices?

Design. The experience-availability axis of §3 predicts that some task slices cannot be addressed by any evolver acting on $\mathcal{H}_t$ alone, because $\mathcal{H}_t$ does not contain the required signal. To test this we identify the Chinese-language platform-data slice of FutureX *a priori* (English-language Wikipedia and mainstream-news priors do not transfer, and the relevant APIs require credentials the evolver cannot obtain from trajectories alone) and compare the full system with and without the two HITL hooks of §3.5: a *taskboard direction* hook between Analyst and Researcher (semantic guidance for novel regimes) and a *builder credential* hook inside Builder execution (resource gate for authentication-required APIs).

Result.

Figure 4: **RQ2 demonstration: a real CTF-Dojo evolution cycle** (provenance: logged cycle, `branch/binary-reversing`). Five panels left-to-right: (1) Analyst reads failing-batch trajectories and writes a new failure regime entry to `task_board.md`; (2) three parallel Researchers explore disjoint hypotheses in isolation and append `works:true/false` rows to `research_log.jsonl`; (3) Builder reads only the `works:true` records, writes `infra/binary_analysis.py`, and updates `architecture.md`; (4) Verifier executes the new pipeline against held-out tasks and reports pass/fail; (5) the originally-failing task is re-solved on the next batch using the routed pipeline. Persistent state files (highlighted) link the phases.

Table 3: HITL impact on the zh-finance slice of FutureX. Gains concentrate on the target slice; performance on other slices is unchanged.

Config	zh-fin.	Other	HITL load
Full system, no HITL	34.6%	52.3%	0 / 4
Full system, + HITL	**53.2%**	52.4%	4 / 4, ∼95 s
Δ	**+18.6**	+0.1	

Table 3 shows that HITL lifts zh-finance accuracy by 18.6 points while moving accuracy on the remaining slices by 0.1 points, within measurement noise. The gain is therefore structurally scoped: it appears only where the evolver's input carries no relevant signal, never as a diffuse "human supervision helps" effect. A typical invocation costs roughly 25 s of human time per cycle (four cycles, about 95 s total), versus zero cost when hooks do not fire.

Figure 7 visualises the targeting pattern: paired bars on the zh-finance slice span an 18.6-point gap, while the same comparison on control slices is within seed noise. Figure 8 shows the two hooks firing in two different cycles of the same FutureX run; they are sequential by construction, since direction must arrive before the Builder has anything to authenticate against.

Finding (additivity). Taken together, the three contributions are approximately additive. The full system's gain over A-Evolve decomposes, within measurement noise, into the sum of the multi-agent-only and navigation-only gains, consistent with the orthogonality of L_{evo} and L_{nav}; HITL contributes an additional increment localised to the experience-insufficient slices. Each component addresses one challenge in isolation, and the composition is what closes the gap to the expert reference.

5 Conclusions

Self-evolving agents deployed on long-horizon, non-stationary, multi-modal task streams exhibit three task-intrinsic failure modes that static benchmarks do not surface: non-monotonic evolution under distribution shift, a construction bottleneck at the prompt-only level, and experience insufficiency in novel regimes. Partitioning the agent's regret into an evolver-class limit $L_{\text{evo}}(\Phi, K)$ and a conditioning limit $L_{\text{nav}}(\varphi)$, with experience availability as a third axis outside the decomposition, yields three composable architectural remedies: a git-backed strategy tree with per-task routing reduces L_{nav}; a four-phase multi-agent evolver with persistent cross-cycle state expands the constructible harness class and reduces L_{evo}; and structurally triggered human-in-the-loop hooks supply direction when history is uninformative. Each is independently effective, and together they close most of the

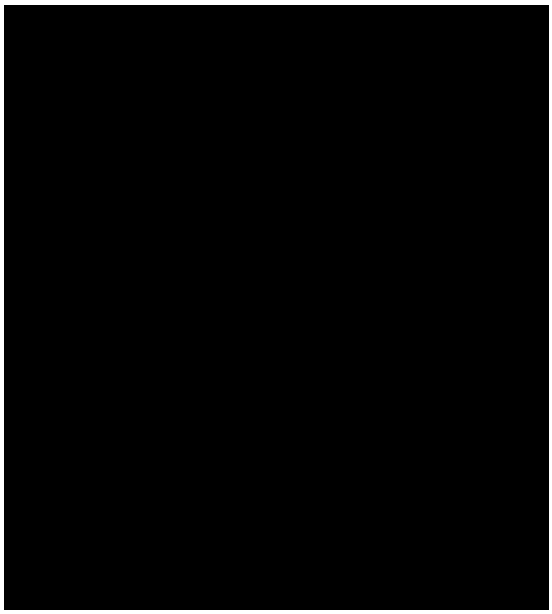

Figure 5: **RQ3 stats.** Three readings on the same evolved tree. *Top*: navigator-minus-unconditional gap versus estimated shift Σ_t, per batch, regression line fit per benchmark (static prediction). *Middle*: per-batch accuracy on the CTF-Dojo stream for unconditional, navigator, and oracle-branch (dynamic prediction). *Bottom*: per-solve harness size in KB; linear evolution grows monotonically while navigation is bounded by the size of the largest single branch.

gap to a hand-designed expert reference on PolyBench, CTF-Dojo, and FutureX.

Open questions. Branch lifecycle (when to promote a specialised branch back into `main` and when to retire a stale one) remains open, as does co-evolution of the navigator with the strategy tree. Automated detection of experience-insufficiency signals without oracle access would remove the one hand-tuned trigger our HITL layer still relies on. Finally, whether the same three challenges and remedies transfer to domains beyond prediction markets, cybersecurity, and forecasting (robotics, biomedical research, long-form writing) is a direction we leave to future work.

Benchmark coverage. Three benchmarks, three domains (prediction markets, cybersecurity, forecasting). The D1/D2/D3 frame's cross-domain generality requires testing on domains we have not yet covered (robotics, biomedical, long-form writing).

Evolver class is not scalar. Our L_{evo} argument is diagnostic, not predictive. Multi-agent's empirical advantage is demonstrated via ablation and artifact-type analysis (RQ2), not a formal class-expansion bound.

HITL cost model. We measure HITL cost by human time per invocation. In a production deployment, organizational cost (specialist availability, SLA) may dominate. RQ4 uses the author as oracle.

Expert reference is author-constructed. The hand-designed harness in Table 2 was built by the authors with hindsight access to the task stream. It is an upper bound, not a competitive SOTA baseline.

Figure 6: **RQ3 demonstration (combined): tree growth and routing trace** (single CTF-Dojo run). *Top*: strategy tree as it grew over cycles. `main` at t=0; `branch/crypto-classical` forked at cycle 2 after the Analyst flagged repeated classical-cipher failures; `branch/binary-reversing` at cycle 4 after stripped-ELF tasks appeared; `branch/web-modern` at cycle 7 after a JWT/SQLi cluster. Each fork is annotated with its trigger. *Bottom*: routing for one task ("RE/2018: stripped x86-64 ELF, find the flag"). F_{Navigate} reads three side-by-side `git show <branch>:README.md` excerpts and emits {`"branch": "binary-reversing", "confidence": 0.83, "reason": "stripped ELF + RE keywords match this branch's tools/registry; main lacks radare2/Ghidra"`}; the solver then checks out the selected branch and produces the flag.

Figure 7: **RQ4 stats.** Targeting pattern of HITL on FutureX: grouped bars over {zh-finance, other} × {no-HITL, +HITL}. The HITL gain is concentrated on the experience-insufficient slice (left pair, +18.6 pts) and is within seed noise on the control slice (right pair, +0.1 pts), the empirical signature of experience insufficiency.

G Ethics statement

H Reproducibility statement

Figure 8: **RQ4 demonstration: two HITL hooks across two cycles** (single FutureX run; cycle indices are run-dependent, typically k=2 and $k+m$=3). *Cycle k, taskboard direction hook (experience insufficiency).* The navigator returns {`"branch": null, "confidence": 0.21`} on three zh-finance tasks; Analyst writes a "novel regime, no prior coverage" tag in `task_board.md`. The HITL prompt fires; a one-line direction ("Chinese-language equity prediction; Eastmoney/Sina Finance are canonical sources; use ZH news + macro indicator joins") is appended and read by the Researchers in the same cycle. *Cycle $k+m$, builder credential hook.* With direction from cycle k, parallel Researchers identify Eastmoney as a viable source. Builder begins constructing `infra/chinese_finance_pipeline.py`; the first execution returns `403 Forbidden`. The credential hook fires; the human supplies `EASTMONEY_API_KEY`; Builder retries; Verifier confirms coverage on held-out zh-finance tasks. The two interventions are mechanistically distinct (semantic direction vs. resource), fire on independent triggers, and never co-occur in a single cycle.